

halfedge：网格修复与洞填充模块设计文档

src/repair.rs

halfedge 库

2026-07-05

目录

1	模块定位	2
1.1	API 总览	2
2	核心算法	2
2.1	remove_face：删除面创建洞	2
2.1.1	流程图	3
2.1.2	边界半边的方向与 next/prev	3
2.1.3	相邻面已删除的处理	3
2.2	fill_hole：洞填充	3
2.2.1	核心设计：不提前删除边界半边	3
2.2.2	流程图	4
2.2.3	清除 next/prev 的必要性	4
2.2.4	twin 配对追踪（以三角形洞为例）	4
2.2.5	多三角形洞的内边处理	4
2.3	fill_all_holes	5
2.4	remove_degenerate_faces	5
2.4.1	数据丢弃警告	5
2.5	非流形检测	5
2.5.1	detect_nonmanifold_edges	5
2.5.2	detect_nonmanifold_vertices	5
2.6	repair_mesh：一键修复 pipeline	6
3	复杂度	7
4	退化守卫	7
5	测试覆盖	8
6	设计权衡	8

1 模块定位

`repair.rs` 提供网格修复操作: 洞填充、面删除、孤立元素清理、非流形检测和一键修复 pipeline。适用于处理导入模型中的拓扑缺陷——缺失面、退化面、孤立顶点等。

1.1 API 总览

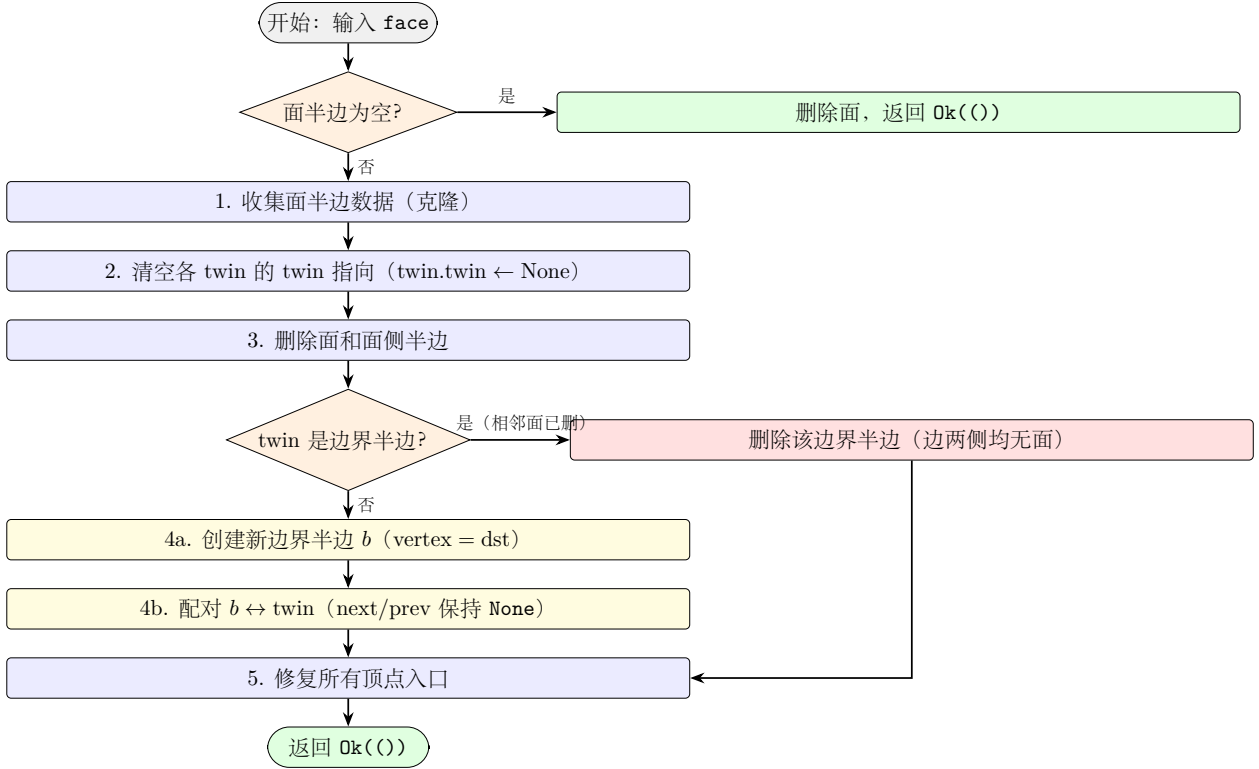
函数	功能	返回值
<code>remove_face</code>	删除面, 创建边界半边环	<code>Result<(), TopErr></code>
<code>fill_hole</code>	填充单个洞	<code>Result<Vec<FaceId>, TopErr></code>
<code>fill_all_holes</code>	填充所有洞	<code>Result<Vec<Vec<FaceId>>, TopErr></code>
<code>remove_isolated_vertices</code>	删除孤立顶点	<code>usize</code>
<code>remove_degenerate_faces</code>	删除退化面	<code>usize</code>
<code>detect_nonmanifold_edges</code>	检测非流形边	<code>Vec<HalfEdgeId></code>
<code>detect_nonmanifold_vertices</code>	检测非流形顶点	<code>Vec<VertexId></code>
<code>repair_mesh</code>	一键修复 pipeline	<code>Result<RepairStats, TopErr></code>

2 核心算法

2.1 `remove_face`: 删除面创建洞

删除面 f 及其关联的面侧半边, 为每条暴露的边创建边界半边, 使洞边界形成完整的半边环供后续 `fill_hole` 使用。

2.1.1 流程图



2.1.2 边界半边的方向与 next/prev

新边界半边 b_i 与被删除的面半边 h_i 同向:

$$b_i.\text{vertex} = h_i.\text{vertex} = \text{dst}_i$$

边界半边不设置 next/prev (保持 None), 因为:

- BoundaryLoop 通过顶点环遍历 (next_boundary_halfedge), 不依赖边界半边的 next/prev;
- 当相邻面被连续删除时, 不维护 next/prev 可避免复杂的链表拼接。

2.1.3 相邻面已删除的处理

当删除面 f_2 的某条面半边的 twin 是边界半边 b (说明相邻面 f_1 已被删除), 直接删除 b 而非创建新边界半边——此时该边两侧均无面, 不存在「洞边界」。

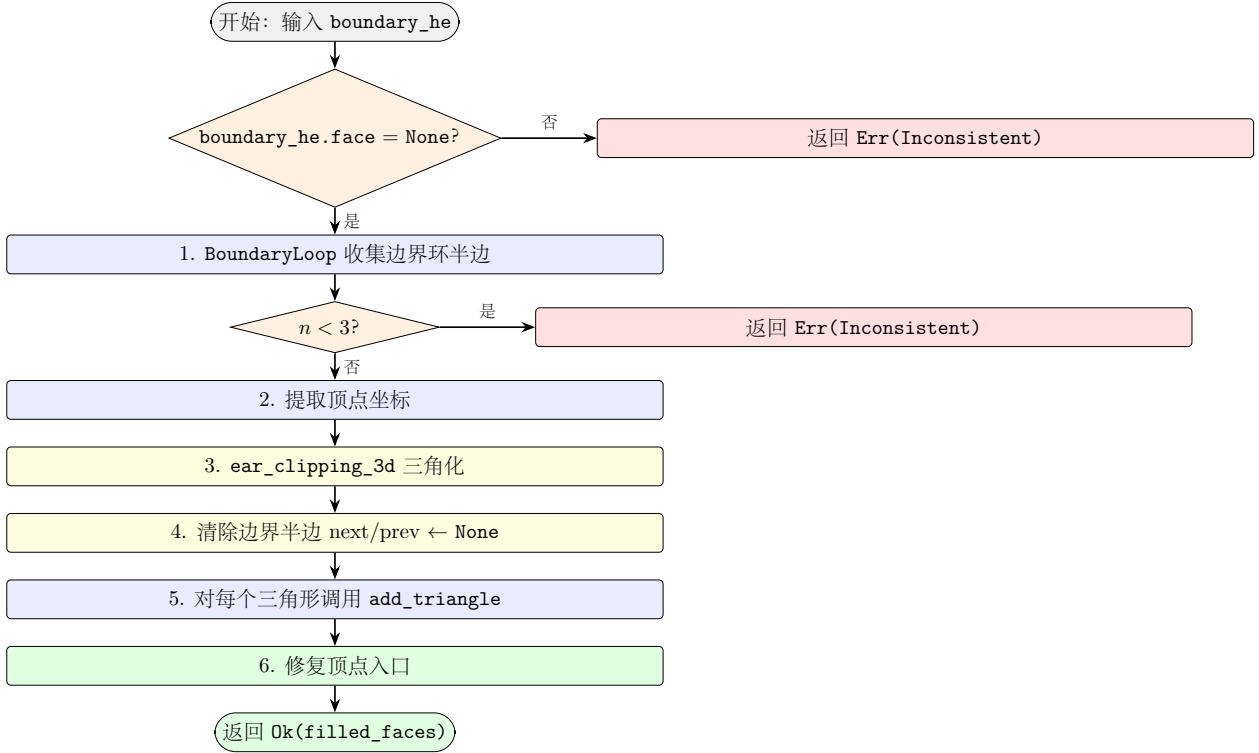
2.2 fill_hole: 洞填充

填充由 boundary_he 指定的单个洞, 返回新创建的面 ID 列表。

2.2.1 核心设计: 不提前删除边界半边

fill_hole 不提前删除边界半边, 而是让 add_triangle 的 twin 配对逻辑自然找到并删除它们。原因: add_triangle 的 twin 配对搜索条件为 $E.\text{vertex} = \text{src} \wedge E.\text{twin}.\text{vertex} = \text{dst} \wedge E.\text{twin}.\text{face} = \text{None}$ 。若提前删除边界半边, 相邻面半边的 twin 变为 None, 配对失败。

2.2.2 流程图



2.2.3 清除 next/prev 的必要性

步骤 4 将边界半边的 next/prev 设为 None。原因：当 add_triangle 删除某个边界半边 b_i 时，环中其他边界半边 b_{i-1} 、 b_{i+1} 的 next/prev 可能仍指向 b_i ，造成悬空指针。清除后 next = None，validate_mesh 跳过该项检查，不报错。

2.2.4 twin 配对追踪（以三角形洞为例）

设洞边界有 3 条半边 $b_0(A \rightarrow B)$ 、 $b_1(B \rightarrow C)$ 、 $b_2(C \rightarrow A)$ ，其 twin 为面半边 f_0 、 f_1 、 f_2 。BoundaryLoop 收集 tip 序列 $[B, C, A]$ ，ear_clipping_3d 返回 $[[0, 1, 2]]$ ，即调用 add_triangle(B, C, A):

新边	搜索条件	匹配	操作
$h_0: B \rightarrow C$	$E.\text{vertex} = B, E.\text{twin}.\text{vertex} = C$	$E = f_1$	删除 b_1 ，配对 $h_0 \leftrightarrow f_1$
$h_1: C \rightarrow A$	$E.\text{vertex} = C, E.\text{twin}.\text{vertex} = A$	$E = f_2$	删除 b_2 ，配对 $h_1 \leftrightarrow f_2$
$h_2: A \rightarrow B$	$E.\text{vertex} = A, E.\text{twin}.\text{vertex} = B$	$E = f_0$	删除 b_0 ，配对 $h_2 \leftrightarrow f_0$

三条边界半边全部被消耗，洞完全填充。

2.2.5 多三角形洞的内边处理

对 $n > 3$ 的洞，ear_clipping_3d 产生多个三角形。首三角形的非边界边（内边）由 add_triangle 创建临时边界半边，后续三角形的 add_triangle 调用会找到该临时边界半边并配对，最终所有边界半边被消耗。

2.3 fill_all_holes

枚举所有边界环，逐个调用 `fill_hole`:

```
1 let loops = boundary_loops(mesh);  
2 for boundary in &loops {  
3     fill_hole(mesh, boundary[0])?;  
4 }
```

2.4 remove_degenerate_faces

用 Shewchuk 鲁棒谓词 `is_triangle_degenerate_3d` 检测三顶点共线的退化面,调用 `remove_face` 删除之。

2.4.1 数据丢弃警告

面删除失败时不再静默，会通过 `eprintln!` 输出警告：

```
[halfedge::remove_degenerate_faces]      N
```

2.5 非流形检测

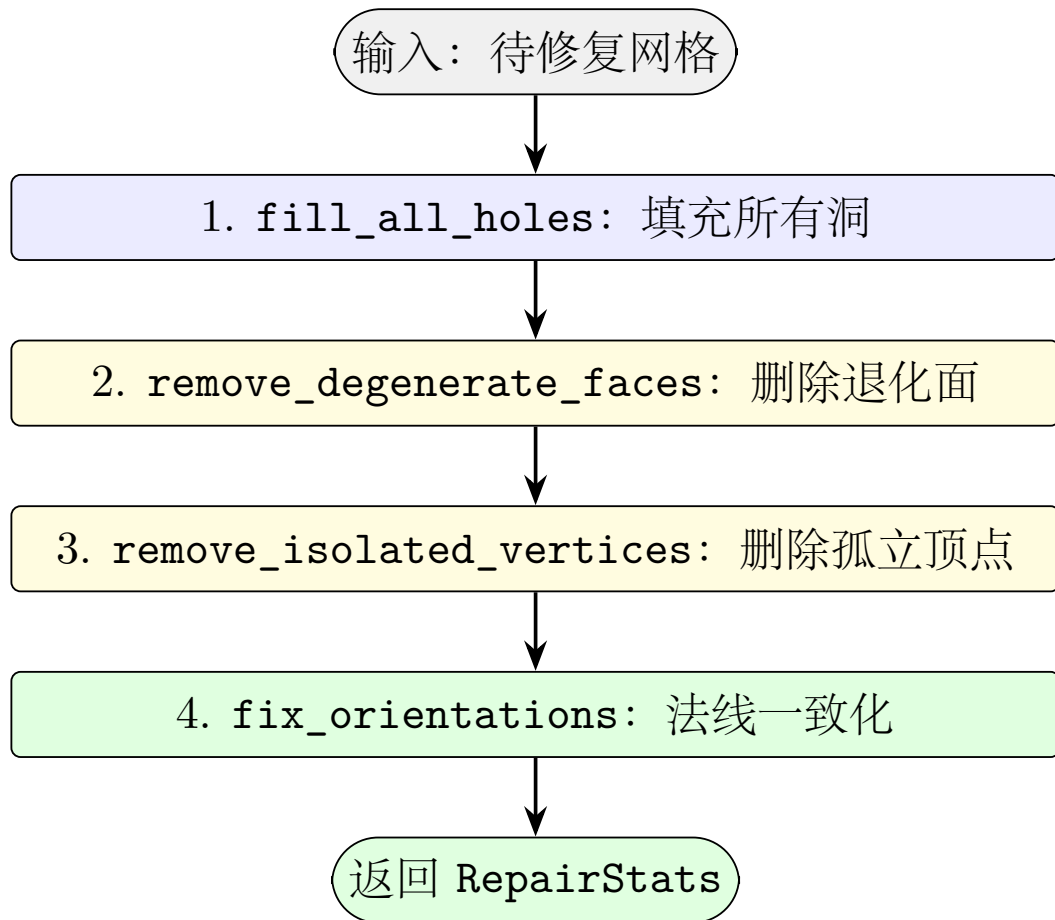
2.5.1 detect_nonmanifold_edges

统计每条无向边被几个面共享。流形网格中每条边至多 2 个面；若 > 2 则为非流形边。

2.5.2 detect_nonmanifold_vertices

对每个顶点检测 `outgoing` 环是否闭合。内部顶点应闭合，边界顶点应为开链。若 `CCW` 和 `CW` 两个方向均不闭合且边界半边数 > 2 ，则为非流形顶点。

2.6 repair_mesh: 一键修复 pipeline



RepairStats 记录各项操作的统计信息：

字段	含义
holes_filled	填充的洞数量
isolated_vertices_removed	删除的孤立顶点数
degenerate_faces_removed	删除的退化面数
faces_flipped	法线一致化翻转的面数

3 复杂度

操作	时间复杂度	备注
<code>remove_face</code>	$O(k + V)$	k = 面边数, V = 顶点修复
<code>fill_hole</code>	$O(n^2 \cdot H)$	n = 洞顶点数, H = 半边总数 (twin 搜索)
<code>fill_all_holes</code>	$O(\sum n_i^2 \cdot H)$	各洞独立处理
<code>remove_isolated_vertices</code>	$O(V)$	线性扫描
<code>remove_degenerate_faces</code>	$O(F \cdot H)$	F = 面数, H = 半边搜索
<code>detect_nonmanifold_edges</code>	$O(H)$	单遍扫描
<code>detect_nonmanifold_vertices</code>	$O(V \cdot \bar{d})$	$\bar{d}$ = 平均度数
<code>repair_mesh</code>	上述之和	pipeline 顺序执行

4 退化守卫

- `fill_hole`: 校验 `boundary_he.face = None`, 否则返回 `Inconsistent`;
- `fill_hole`: 边界环顶点数 < 3 时返回 `Inconsistent`;
- `remove_face`: 面半边为空时直接删除面并返回;
- `remove_face`: 相邻面已删除时 (twin 是边界半边), 直接删除该边界半边而非创建新边界;
- `remove_degenerate_faces`: 使用 Shewchuk 鲁棒谓词, 避免浮点阈值误判;
- `detect_nonmanifold_vertices`: 使用 `halfedge_count + 1` 作为最大迭代次数, 防止死循环;
- 所有顶点入口修复使用 `find_outgoing_halfedge` 确保引用有效。

5 测试覆盖

测试名	验证点
<code>fill_hole_basic</code>	单面洞填充后无残留边界
<code>fill_all_holes_icosphere</code>	icosphere 单面洞一键填充
<code>fill_hole_on_closed_mesh_returns_empty</code>	闭合网格无洞
<code>fill_hole_non_boundary_is_error</code>	非边界半边报错
<code>remove_isolated_vertices_basic</code>	删除无关联顶点
<code>remove_face_creates_boundary</code>	删除面后产生 3 条边界半边
<code>remove_face_and_fill_restores</code>	删除 + 填充恢复面数
<code>detect_nonmanifold_edges_closed_mesh</code>	闭合无非流形边
<code>repair_mesh_basic</code>	一键修复统计正确
<code>fill_hole_quad_hole</code>	grid 边界洞填充
<code>remove_and_fill_multiple_faces</code>	删除 2 面 + 填充
<code>remove_degenerate_faces_on_good_mesh</code>	好网格无退化面

全模块共 12 个单元测试；项目整体 `cargo test` 共 450 个用例。

6 设计权衡

- **不删除边界半边 vs 提前删除**: `fill_hole` 保留边界半边让 `add_triangle` 自然配对，而非提前删除后手动重建 `twin`。后者需手动建立 `twin` 关系，逻辑更复杂且易遗漏。
- **边界半边 `next/prev = None`**: `BoundaryLoop` 用顶点环遍历，不依赖边界半边的 `next/prev`。设为 `None` 避免连续删除相邻面时的链表拼接问题，`validate_mesh` 对 `next = None` 跳过检查。
- **`ear_clipping_3d` 而非扇形三角化**: 对凹多边形洞，扇形三角化可能产生翻转面；`ear_clipping_3d` 投影到主平面后使用耳切法，能正确处理凹洞。
- **非流形检测 vs 修复**: 当前仅提供检测，不自动修复非流形拓扑。修复策略取决于应用场景，过早修复可能破坏用户意图。